

Voting Machine Hashcode Testing: Unsurprisingly insecure, and surprisingly insecure

 blog.citp.princeton.edu/2021/03/05/voting-machine-hashcode-testing-unsurprisingly-insecure-and-surprisingly-insecure

Andrew Appel

March 5, 2021

By Andrew Appel and Susan Greenhalgh

The accuracy of a voting machine is dependent on the software that runs it. If that software is corrupted or hacked, it can misreport the votes. There is a common assumption that we can check the legitimacy of the software that is installed by checking a “hash code” and comparing it to the hash code of the authorized software. In practice the scheme is supposed to work like this: Software provided by the voting-machine vendor examines all the installed software in the voting machine, to make sure it’s the right stuff.

There are some flaws in this concept: it’s hard to find “all the installed software in the voting machine,” because modern computers have many layers underneath what you examine. But mainly, if a hacker can corrupt the vote-tallying software, perhaps they can corrupt the hash-generating function as well, so that whenever you ask the checker “does the voting machine have the right software installed,” it will say, “Yes, boss.” Or, if the hasher is designed not to say “yes” or “no,” but to report the hash of what’s installed, it can simply report the hash of what’s *supposed* to be there, not what’s *actually* there. For that reason, election security experts never put much reliance in this hash-code idea; instead they insist that you can’t fully trust what software is installed, so you must achieve election integrity by doing recounts or risk-limiting audits of the paper ballots.

But you might have thought that the hash-code could at least help protect against accidental, nonmalicious errors in configuration. You would be wrong. It turns out that ES&S has bugs in their hash-code checker: **if the “reference hashcode” is completely missing, then it’ll say “yes, boss, everything is fine” instead of reporting an error.** It’s simultaneously **shocking** and **unsurprising** that ES&S’s hashcode checker could contain such a blunder **and** that it would go unnoticed by the U.S. Election Assistance Commission’s federal certification process. It’s unsurprising because testing naturally tends to focus on “does the system work right when used as intended?” Using the system in unintended ways (which is what hackers would do) is not something anyone will notice.

Until somebody **does** notice. In this case, it was the State of Texas’s voting-machine examiner, Brian Mechler. In his report dated September 2020 he found this bug in the hash-checking script supplied with the ES&S EVS 6.1.1.0 election system (for the ExpressVote

touch-screen BMD, the DS200 in-precinct optical scanner, the DS450 and DS850 high-speed optical scanners, and other related voting machines). (Read Section 7.2 of [Mr. Mechler's report](#) for details).

We can't know whether that bug was intentional or not. Either way, it's certainly convenient for ES&S, because it's one less hassle when installing firmware upgrades. (Of course, it's one less hassle for potential hackers, too.)

Another gem in Mr. Mechler's report is in Section 7.1, in which he reveals that *acceptance testing* of voting systems is done by the *vendor*, not by the customer. Acceptance testing is the process by which a customer checks a delivered product to make sure it satisfies requirements. To have the vendor do acceptance testing pretty much defeats the purpose.

When the Texas Secretary of State learned that their vendor was doing the acceptance testing themselves, the SoS's Election Division took an action "to work with ES&S and their Texas customers to better define their roles and responsibilities with respect to acceptance testing," according to the report. They may encounter a problem, though: the ES&S sales contract specifies that ES&S must perform the acceptance testing, or they will void your warranty (see clause 7b).

There's another item in Mr. Mechler's report, Section 7.3. The U.S. Election Assistance Commission requires that "The vendor shall have a process to verify that the correct software is loaded, that there is no unauthorized software, and that voting system software on voting equipment has not been modified, using the reference information from the [National Software Reference Library] or from a State designated repository. The process used to verify software should be possible to perform without using software installed on the voting system." This requirement is usually interpreted to mean, "check the hash code of the installed software against the reference hash code held by the EAC or the State."

But ES&S's hash-checker doesn't do that at all. Instead, ES&S instructs its techs to create some "golden" hashes from the first installation, then subsequently check the hash code against these. So whatever software was first installed gets to be "golden", regardless of whether it's been approved by the EAC or by the State of Texas. This design decision was probably a convenient shortcut by engineers at ES&S, but it directly violates the EAC's rules for how hash-checking is supposed to work.

So, what have we learned?

We already knew that hash codes can't protect against hackers who install vote-stealing software, because the hackers can also install software that lies about the hash code. But now we've learned that hash codes are even more useless than we might have thought. This voting-machine manufacturer

- has a hash-code checker that erroneously reports a match, even when you forget to tell it what to match against;
- checks the hash against what was first installed, not against the authorized reference that they're supposed to;
- and the vendor insists on running this check itself — not letting the customer do it — otherwise the warranty is voided.

As a bonus we learned that the EAC certifies voting systems without checking if the validation software functions properly.

Are we surprised? You know: *fool me once, shame on you; fool me twice, shame on me*. Every time that we imagine that a voting-machine manufacturer might have sound cybersecurity practices, it turns out that they've taken shortcuts and they've made mistakes. In this, voting-machine manufacturers are no different from any other makers of software. There's lots of insecure software out there made by software engineers who cut corners and don't pay attention to security, and why should we think that voting machines are any different?

So if we want to trust our elections, we should vote on hand-marked paper ballots, counted by optical scanners, and recountable by hand. Those optical scanners are pretty accurate when they haven't been hacked — even the ES&S DS200 — and it's impractical to count all the ballots without them. But we should always check up on the machines by doing random audits of the paper ballots. And those audits should be “strong” enough — that is, use good statistical methods and check *enough* of the ballots — to catch the mistakes that the machines might make, if the machines make mistakes (or are hacked). The technical term for those “strong enough” audits is *Risk-Limiting Audit*.

Andrew W. Appel is Professor of Computer Science at Princeton University.

Susan Greenhalgh is Senior Advisor on Election Security at Free Speech For People.